

Contents

I	Preliminaries	
1	Foundations of Machine Learning	17
1.1	Math Basics	18
1.1.1	Linear Algebra	18
1.1.2	Probability and Statistics	22
1.2	Designing a Text Classifier	25
1.2.1	Problem Formulation	26
1.2.2	Documents as Feature Vectors	28
1.2.3	Linear Classifiers	29
1.2.4	Generative vs Discriminative	30
1.2.5	OOV Words and Smoothing	34
1.3	General Problems	36
1.3.1	Supervised and Unsupervised Models	36
1.3.2	Inductive Bias	37
1.3.3	Non-linearity	39
1.3.4	Training and Loss Functions	41
1.3.5	Overfitting and Underfitting	46
1.3.6	Prediction and Inference	48
1.4	Model Selection and Evaluation	49
1.4.1	Strategies for Model Selection	50
1.4.2	Training, Validation and Test Data	55
1.4.3	Performance Measure	56
1.4.4	Significance Tests	56
1.5	NLP Tasks as ML Tasks	58
1.5.1	Classification	58
1.5.2	Sequence Labeling	59
1.5.3	Language Modeling/Word Prediction	60
1.5.4	Sequence Generation	61
1.5.5	Tree Generation	62
1.5.6	Relevance Modeling	64

1.5.7	Linguistic Alignment	65
1.5.8	Information Extraction	66
1.5.9	Remarks	67
1.6	Summary	67
2	Foundations of Neural Networks	69
2.1	Multi-layer Neural Networks	69
2.1.1	Single-layer Perceptrons	69
2.1.2	Stacking Multiple Layers	71
2.1.3	Computation Graphs	73
2.2	Example: Neural Language Modeling	76
2.3	Basic Model Architectures	81
2.3.1	Recurrent Units	81
2.3.2	Convolutional Units	83
2.3.3	Gate Units	86
2.3.4	Normalization (Standardization) Units	86
2.3.5	Residual Units	87
2.4	Training Neural Networks	88
2.4.1	Gradient Descent	89
2.4.2	Batching	93
2.4.3	Parameter Initialization	94
2.4.4	Learning Rate Scheduling	95
2.5	Regularization Methods	98
2.5.1	Norm-based Penalties	98
2.5.2	Dropout	99
2.5.3	Early Stopping	101
2.5.4	Smoothing Output Probabilities	102
2.5.5	Training with Noise	104
2.6	Unsupervised Methods and Auto-encoders	106
2.6.1	Auto-encoders with Explicit Regularizers	109
2.6.2	Denoising Auto-encoders	112
2.6.3	Variational Auto-encoders	113
2.7	Summary	118

II

Basic Models

3	Words and Word Vectors	121
3.1	Tokenization	121
3.1.1	Tokenization via Rules and Heuristics	122
3.1.2	Tokenization as Language Modeling	124

3.1.3	Tokenization as Sequence Labeling	127
3.1.4	Learning Subwords	128
3.2	Vector Representation for Words	135
3.2.1	One-hot Representation	135
3.2.2	Distributed Representation	136
3.2.3	Compositionality and Contextuality	138
3.3	Count-based Models	140
3.3.1	Co-occurrence Matrices	140
3.3.2	TF-IDF	143
3.3.3	Low-Dimensional Models	144
3.4	Inducing Word Embeddings from NLMs	150
3.5	Word Embedding Models	152
3.5.1	Word2Vec	152
3.5.2	GloVe	154
3.5.3	Remarks	158
3.6	Evaluating Word Embeddings	160
3.6.1	Extrinsic Evaluation	161
3.6.2	Intrinsic Evaluation	161
3.6.3	Visualization	165
3.7	Summary	165
4	Recurrent and Convolutional Sequence Models	169
4.1	Problem Statement	170
4.2	Recurrent Models	171
4.2.1	RNN-based Language Models	171
4.2.2	Training	173
4.2.3	Layer Stacking	176
4.2.4	Bi-directional Models	178
4.3	Memory	179
4.3.1	Memory as A System	180
4.3.2	Long Short-Term Memory	181
4.3.3	Gated Recurrent Units	183
4.4	Convolutional Models	185
4.4.1	Convolution	185
4.4.2	CNNs for Sequence Modeling	189
4.4.3	Handling Positional Information	191
4.5	Examples	196
4.5.1	Text Classification	196
4.5.2	End-to-End Speech Recognition	198
4.5.3	Sequence Labeling with LSTM and Graphical Models	201

4.5.4	Hybrid Models for Language Modeling	205
4.6	Summary	207
5	Sequence-to-Sequence Models	211
5.1	Sequence-to-Sequence Problems	212
5.2	The Encoder-Decoder Architecture	214
5.2.1	Encoding and Decoding	214
5.2.2	Example: Neural Machine Translation	216
5.3	The Attention Mechanism	219
5.3.1	A Basic Model	219
5.3.2	The QKV Attention	223
5.3.3	Multi-head Attention	226
5.3.4	Multi-layer Attention	229
5.3.5	Remarks	231
5.4	Search	236
5.4.1	The Length Problem	236
5.4.2	Pruning and Beam Search	240
5.4.3	Online Search	248
5.4.4	Exact Search	252
5.4.5	Differentiable Search	254
5.4.6	Hypothesis Diversity	256
5.4.7	Combining Multiple Models	258
5.4.8	More Search Objectives	260
5.5	Summary	263
6	Transformers	267
6.1	The Basic Model	267
6.1.1	The Transformer Architecture	267
6.1.2	Positional Encoding	271
6.1.3	Multi-head Self-attention	272
6.1.4	Layer Normalization	274
6.1.5	Feed-forward Neural Networks	275
6.1.6	Decoder-side Attention	276
6.1.7	Training and Inference	279
6.2	Syntax-aware Models	281
6.2.1	Syntax-aware Input and Output	282
6.2.2	Syntax-aware Attention Models	283
6.2.3	Multi-branch Models	285
6.2.4	Multi-scale Models	288
6.2.5	Transformers as Syntax Learners	289

6.3	Improved Architectures	293
6.3.1	Locally Attentive Models	293
6.3.2	Deep Models	297
6.3.3	Numerical Method-Inspired Models	303
6.3.4	Wide Models	306
6.4	Efficient Models	310
6.4.1	Sparse Attention	310
6.4.2	Recurrent and Memory Models	315
6.4.3	Low-dimensional Models	319
6.4.4	Parameter and Activation Sharing	325
6.4.5	Alternatives to Self-Attention	326
6.4.6	Conditional Computation	333
6.4.7	Model Transfer and Pruning	338
6.4.8	Sequence Compression	340
6.4.9	High Performance Computing Methods	342
6.5	Applications	345
6.5.1	Language Modeling	345
6.5.2	Text Encoding	346
6.5.3	Speech Translation	348
6.5.4	Vision Models	351
6.5.5	Multimodal Models	352
6.6	Summary	353

III

Large Language Models

7	Pre-training	363
7.1	Pre-training in NLP	364
7.1.1	Unsupervised, Supervised and Self-supervised Pre-training	364
7.1.2	Adapting Pre-trained Models	366
7.2	Self-supervised Pre-training Tasks	369
7.2.1	Decoder-only Pre-training	370
7.2.2	Encoder-only Pre-training	371
7.2.3	Encoder-Decoder Pre-training	378
7.2.4	Comparison of Pre-training Tasks	384
7.3	Example: BERT	386
7.3.1	The Standard Model	386
7.3.2	More Training and Larger Models	391
7.3.3	More Efficient Models	391
7.3.4	Multilingual Models	392

7.4	Applying BERT Models	394
7.5	Summary	399
8	Generative Models	401
8.1	A Brief Introduction to LLMs	402
8.1.1	Decoder-only Transformers	403
8.1.2	Training LLMs	406
8.1.3	Fine-tuning LLMs	407
8.1.4	Aligning LLMs with the World	413
8.1.5	Prompting LLMs	418
8.2	Training at Scale	423
8.2.1	Data Preparation	423
8.2.2	Model Modifications	425
8.2.3	Distributed Training	428
8.2.4	Scaling Laws	431
8.3	Long Sequence Modeling	434
8.3.1	Optimization from an HPC Perspective	435
8.3.2	Efficient Architectures	436
8.3.3	Cache and Memory	439
8.3.4	Sharing across Heads and Layers	448
8.3.5	Position Extrapolation and Interpolation	451
8.3.6	Remarks	462
8.4	Summary	464
9	Prompting	467
9.1	General Prompt Design	468
9.1.1	Basics	468
9.1.2	In-context Learning	471
9.1.3	Prompt Engineering Strategies	473
9.1.4	More Examples	478
9.2	Advanced Prompting Methods	488
9.2.1	Chain of Thought	488
9.2.2	Problem Decomposition	490
9.2.3	Self-refinement	498
9.2.4	Ensembling	504
9.2.5	RAG and Tool Use	508
9.3	Learning to Prompt	513
9.3.1	Prompt Optimization	514
9.3.2	Soft Prompts	517
9.3.3	Prompt Length Reduction	528
9.4	Summary	529

10	Alignment	531
10.1	Overview of LLM Alignment	532
10.2	Instruction Alignment	533
10.2.1	Supervised Fine-tuning	534
10.2.2	Fine-tuning Data Construction	539
10.2.3	Fine-tuning with Less Data	544
10.2.4	Instruction Generalization	545
10.2.5	Weak-to-Strong Generalization	547
10.3	Human Preference Alignment: RLHF	551
10.3.1	Basics of Reinforcement Learning	551
10.3.2	Training Reward Models	558
10.3.3	Training LLMs	561
10.4	Improved Human Preference Alignment	566
10.4.1	Better Reward Modeling	566
10.4.2	Direct Preference Optimization	573
10.4.3	Automatic Preference Data Generation	576
10.4.4	Step-by-step Alignment	578
10.4.5	Inference-time Alignment	581
10.5	Summary	582
11	Inference	585
11.1	Prefilling and Decoding	586
11.1.1	Preliminaries	586
11.1.2	A Two-phase Framework	591
11.1.3	Decoding Algorithms	594
11.1.4	Evaluation Metrics for LLM Inference	605
11.2	Efficient Inference Techniques	606
11.2.1	Prefix Caching	606
11.2.2	Batching	607
11.2.3	Parallelization	616
11.2.4	Remarks	617
11.3	Inference-time Scaling	619
11.3.1	Context Scaling	620
11.3.2	Search Scaling	620
11.3.3	Output Ensembling	621
11.3.4	Thinking-Path Generation and Verification	622
11.4	Summary	630